Azure Data Factory

Document ID	DOC002
Project	Azure Data Factory
Category	Data Engineering
Batch	batch_1

Purpose: Original educational notes created as a searchable PDF corpus for a portfolio-scale incremental RAG pipeline. The material is designed to contain meaningful technical sections that naturally produce multiple retrieval chunks.

ADF Overview

Azure Data Factory is a cloud data integration and orchestration service. It is commonly used to move data between sources and destinations and to coordinate data processing activities. ADF solutions are organized around pipelines, activities, datasets, linked services, integration runtimes, parameters, variables, and triggers.

Practical note. For data engineering work, the design should make processing boundaries explicit, preserve traceability, and avoid unnecessary recomputation when only a subset of the source has changed.

Pipelines and Activities

A pipeline is a logical container for a workflow. Activities inside a pipeline perform work such as copying data, executing another pipeline, looking up configuration, iterating through files, invoking a stored procedure, or running a transformation. Dependencies between activities control execution order and can respond to success, failure, completion, or skip conditions.

Practical note. For data engineering work, the design should make processing boundaries explicit, preserve traceability, and avoid unnecessary recomputation when only a subset of the source has changed.

Copy Activity and Integration Runtime

Copy Activity moves data from a source to a sink. The Integration Runtime provides the compute and network bridge used by ADF activities. Azure Integration Runtime supports cloud data movement and dispatch. Self-hosted Integration Runtime is commonly used when ADF must access resources in private networks or on-premises environments. Azure-SSIS Integration Runtime supports SSIS package execution.

Practical note. For data engineering work, the design should make processing boundaries explicit, preserve traceability, and avoid unnecessary recomputation when only a subset of the source has changed.

Parameterization and Dynamic Content

Parameters make pipelines, datasets, and linked services reusable. Instead of creating a separate pipeline for every table or file, values such as table name, folder path, file name, and load type can be passed at runtime. ADF expressions and dynamic content combine parameters, variables, activity outputs, and system values to construct runtime behavior.

Practical note. For data engineering work, the design should make processing boundaries explicit, preserve traceability, and avoid unnecessary recomputation when only a subset of the source has changed.

Incremental Loading

Incremental loading processes data that is new or changed since a previous successful run. A common relational pattern stores a watermark such as the latest processed timestamp or increasing identifier. A Lookup activity can retrieve the old watermark and another query can determine the new boundary. Copy Activity then reads only rows between those values. After successful processing, the stored watermark is updated.

Practical note. For data engineering work, the design should make processing boundaries explicit, preserve traceability, and avoid unnecessary recomputation when only a subset of the source has changed.

Control Flow and Monitoring

Get Metadata can inspect files and folders, ForEach can iterate through collections, If Condition can branch based on an expression, and Execute Pipeline can support modular parent-child designs. Retry policies can handle temporary failures. Audit tables can record run identifiers, row counts, timestamps, and statuses. Scheduled triggers can start pipelines at defined times, while event-based patterns can react to supported storage events.

Practical note. For data engineering work, the design should make processing boundaries explicit, preserve traceability, and avoid unnecessary recomputation when only a subset of the source has changed.

End of document.